AI ARCHITECTURE WEEKLY

# 5 CLAUDE.md mistakes with full fix examples

A 12-page guide for senior devs working with AI tooling daily. Production templates you can copy.

**Alex Rogov**

JS/TS Architect — Amsterdam · alexrogov.substack.com · 2026

# Why this PDF exists

The newsletter version of this material is around 1700 words. It tells you what the five mistakes are and what to fix. This PDF goes further: for each mistake, you get the full template I use in production, the exact CLAUDE.md sections to copy, and the failure modes I've seen these fixes prevent.

**Read time:** 15-20 minutes. **Apply time:** about an hour per project.

If you want one of these every Monday — same depth, different topic — the newsletter is at alexrogov.substack.com.

# Mistake 1 — Treating CLAUDE.md as a prompt

**Symptom you'll recognize:** the file grows every session. You add a rule because Claude did something stupid yesterday. Next week another rule for another stupid thing. The file is now 800 lines and Claude still ignores half of it.

**Why this happens:** prompts are transient — single-task, throw-away. CLAUDE.md is the opposite — it's the document a senior hire would read on day one to ship code without breaking things on day two. When you treat it like a prompt, it accumulates patches that don't belong in a permanent doc.

## The test I use

Before adding any line to CLAUDE.md, ask: would this rule survive a year of refactors?

- ✅ "Imports flow inward only: interfaces → application → domain"
- ❌ "Don't use `useEffect` in `OrderList.tsx`"

The first is architecture. The second is a patch — it belongs in a commit message, not a permanent doc.

## Concrete fix

Audit your current CLAUDE.md. For each rule, classify it:

| Type | Where it belongs |
|---|---|
| Architectural decision (folder structure, dependency rules, type system policy) | CLAUDE.md |
| Stable convention (naming, error handling pattern, test layout) | CLAUDE.md |
| Patch for one bug ("don't do X in file Y") | Commit message |
| Reaction to a single Claude failure | Throw away — model will adapt |
| Style preference (tabs/spaces, single-quote/double) | linter/formatter config |

When I did this on a 800-line CLAUDE.md, it shrank to 247 lines. Claude's accuracy went up, not down.

## Production template — the only structure CLAUDE.md needs

```
# Project Name

## Stack
[2–3 lines: framework, language version, key libraries]

## Project layout
[8–15 lines: top-level folders + their purpose + import rules]

## Why we made specific decisions
[5–10 short sections with **why** behind major choices: TypeScript strict,
Clean Architecture, DDD, etc. ~5–15 lines each]

## Patterns we use
[3–5 examples of "the way we do X" with code snippets]

## Pitfalls / things that broke us
[5–10 items: deprecations, hidden gotchas, legacy folders to avoid]

## What this codebase is NOT
[Explicit non-goals — saves arguments]
```

Anything else is leakage. If a thought doesn't fit one of these six sections, it doesn't belong in CLAUDE.md.

# Mistake 2 — Listing rules instead of teaching context

**Symptom:** five sessions in, Claude breaks half your rules. You're frustrated. You add MORE rules.

**Why this happens:** rules without context are noise. The model technically can follow each rule in isolation, but it doesn't know which one to prioritize when they conflict. And rules always conflict in real code.

## Concrete example

**Bad — rule-only:**

```
- Never use `any`
- Always use TypeScript strict mode
- Always write tests
- Use functional components only
- Don't use `useEffect` for data fetching
```

When Claude hits a generic case where typing is hard, it has no reason to spend extra effort. It defaults to the path of least resistance.

**Good — context-rich:**

```
## Why we avoid `any`

We migrated 200K lines from JS to strict TS in 2024.
Every `any` we left in became a runtime bug within 6 months.
The cost of strict types is 3x slower initial typing.
The cost of `any` is one production incident every 2 sprints.

If you genuinely need an escape hatch, use `unknown` and narrow.
Never `any`. There are no exceptions in this repo.
```

Same rule. Now Claude knows *why* and how much it matters. When a tricky generic case arrives, it spends the extra effort because it understands the cost.

## Production template — how to write any rule

```
## Why we [rule]

[What changed/decided in plain language — 1–2 sentences]
[The cost of NOT following the rule — 1–2 sentences with real numbers]
[The cost of following the rule — 1 sentence — be honest]

[Exact rule + named alternative if there is one]
```

Apply this to your top 5 rules. The model's behavior on edge cases will change.

## Failure mode this prevents

Without context, Claude will **silently optimize for ease**. With context, it optimizes for cost. Real example: in a project where I switched from rule-only to context-rich, Claude went from generating `any` types ~3 times per session to ~0 times per session. Same model. Same task complexity. Different CLAUDE.md.

# Mistake 3 — Skipping the project map

**Symptom:** Claude creates files in the wrong directories. Adds a hook to `utils/` instead of `hooks/`. Generates a duplicate of an existing helper. Imports from a deep path when there's a barrel export it didn't see.

**Why this happens:** the model doesn't have your folder structure burned into its context. It's guessing based on file naming conventions it's seen in millions of other projects. None of which are yours.

## Concrete fix

Add an explicit map. Eight lines is enough.

```
## Project layout

src/
├── domain/        — pure business logic, no framework imports
├── application/   — use cases, orchestrates domain
├── infrastructure/— external services, db, APIs
└── interfaces/    — entry points (HTTP, CLI, jobs)

Rules:
- Imports flow inward only: interfaces → application → domain
- domain/ has zero dependencies. If you're tempted to import from
  there into utils, you're putting the wrong thing in domain/
- Each layer has a CLAUDE.md sub-file with its conventions
```

## Production template — adapt to your stack

For a Next.js + NestJS monorepo:

```
## Project layout

apps/
├── web/           — Next.js frontend (App Router)
├── api/           — NestJS backend (Clean Architecture)
└── mobile/        — React Native (planned)

packages/
├── shared/        — types + zod schemas shared across apps
├── ui/            — design system components
└── config/        — eslint, tsconfig, prettier

Conventions:
- packages/shared has ZERO runtime dependencies (only zod, @types/*)
- apps/web imports from packages/* but NEVER from apps/api
- New types: declare in packages/shared, not in app-local files
- Database: only apps/api touches /infrastructure/database
```

For a single React app:

```
## Project layout

src/
├── components/    — UI primitives (button, card, modal)
├── features/      — feature folders (auth/, dashboard/, billing/)
│                    each has its own components/, hooks/, types.ts
├── hooks/         — cross-feature shared hooks (useDebounce, useApiQuery)
├── lib/           — pure utility functions
└── pages/         — Next.js routing only

Rules:
- features/* import from components/, hooks/, lib/ — never from each other
- New page = features/<name>/page.tsx + import to pages/
- Don't add files to /src/ root — pick a folder
```

## Why this matters more than people think

Without a map, Claude treats your codebase like a generic React/Node app. With one, it treats your codebase like *the* codebase. The difference shows up week three, when you ask for a new feature and the agent puts files where they belong without asking.

# Mistake 4 — Generic patterns instead of project-specific ones

**Symptom:** your CLAUDE.md says "use functional components." Claude does. Then it imports `useState` and `useEffect` and writes data fetching from scratch — even though you have a `useApiQuery` hook five lines into your codebase.

**Why this happens:** the model knows React. It does not know your project's flavor of React. Without examples of your specific patterns, it defaults to generic 2022-era React patterns that don't fit your stack.

## Concrete fix

Show — don't just tell.

**In CLAUDE.md:**

> ### How we handle data fetching
>
> We use a project-specific `useApiQuery` hook ( `src/hooks/use-api-query.ts` ). It wraps TanStack Query with our auth, error handling, and retry policy.
>
> Don't write fetch logic from scratch. Don't import `useQuery` directly unless you're inside `use-api-query.ts` itself.

```
// Example usage:
import { useApiQuery } from '@/hooks/use-api-query'

const { data, isLoading } = useApiQuery({
  key: ['users', userId],
  endpoint: `/api/users/${userId}`,
})
```

## Which patterns to document

Write examples for the patterns that come up *every week*. Not edge cases. The 80/20 list:

1. **Data fetching** — your useQuery wrapper, error handling, retry policy
2. **Forms** — your form library + validation pattern (zod + react-hook-form?)
3. **Auth** — how to get the current user, how to gate routes

4. **Error boundaries** — where they wrap, what they show
5. **Logging** — your logger setup, what to log, what NOT to log

If a pattern shows up in five PRs a month, it goes in CLAUDE.md.

## Production template — the standard pattern block

```
## How we [thing]

We use [library/pattern] via [our wrapper] (`path/to/file.ts`).
[1-2 lines: what's special about our wrapper]

Don't [thing 1]. Don't [thing 2] unless [exception].

// Example:
[real working code that fits your project]
```

Three to five of these blocks. That's enough.

## What changes when you do this

Without project-specific patterns, Claude writes code that fits a generic codebase. With them, it writes code that fits *your* codebase. Code review velocity goes up. Senior reviewers stop tagging "use X helper" on every PR.

# Mistake 5 — No "things that broke us" section

**Symptom:** every team has tribal knowledge. The library version with a memory leak. The util that looks generic but has business logic baked in. The folder that nobody touches because of a forgotten migration.

This knowledge lives in Slack threads. In PR comments. In the heads of two seniors who've been on the team longest.

**The model can't ask Slack.**

## What happens without a Pitfalls section

Claude confidently rediscovers every footgun your team already learned about. It uses the deprecated method. It reaches into the cursed folder. It picks the abstraction that looked clean but caused three incidents last year.

## Concrete fix

Add a Pitfalls section. Five items minimum.

```
## Pitfalls / things that broke us

- `lodash.merge` mutates targets in v4.17.20. Use `deepmerge` instead.
- `src/legacy-api/` is being phased out. Don't add new code there.
  All new endpoints go in `src/api/`. The two folders look similar
  but legacy-api uses a different auth flow.
- Don't use `Date.now()` for IDs. Tests run in parallel and you'll
  get collisions. Use `crypto.randomUUID()`.
- TanStack Query's `keepPreviousData` is removed in v5. We use
  `placeholderData: keepPreviousData` import instead.
- Our `Money` type is in cents (integer), never decimal. If you see
  a decimal, it's a bug — convert immediately at the boundary.
```

## How to source these

Three ways:

1. **PR archaeology**: search closed PRs for "fix:", "hotfix", "revert" in the last 6 months. The bug was something someone already learned. Document it.
2. **Senior interview**: ask the two seniors who've been there longest "what's the one thing that catches new hires every time?" Write down their answers.

3. **Incident retros**: any post-mortem with "we should add X to onboarding" — that X goes in Pitfalls.

## Why this section pays the most

Each pitfall costs a real engineer real hours to discover. Once written down, it costs the agent zero seconds to avoid. The ROI on a 10-line Pitfalls section is higher than on most architectural decisions.

**Not every CLAUDE.md needs one. But every CLAUDE.md older than three months has one waiting to be written.**

## Putting it together — a CLAUDE.md audit checklist

When auditing an existing CLAUDE.md, walk this list:

- ☐ **Length check**: under 300 lines. If longer, you've accumulated patches that belong elsewhere.
- ☐ **Project map exists**: explicit folder structure with import rules. 5-15 lines.
- ☐ **At least 3 "Why we [decision]" sections**: TypeScript strict, architecture style, testing strategy.
- ☐ **3-5 pattern examples**: data fetching, forms, auth — the patterns that show up every PR.
- ☐ **Pitfalls section with 5+ items**: real failures from real PRs.
- ☐ **No patches**: no "don't do X in file Y" rules. Those go in commit messages.
- ☐ **Survives the year test**: every rule would still be valid after a year of refactors.

Run this audit on one project this week. The diff in Claude's behavior is visible within three days.

## What's next

This PDF is companion material to the **AI Architecture Weekly** newsletter.

Every Monday at 09:00 CET, you get one specific thing I've learned working as a JS/TS architect with AI in the loop:

- CLAUDE.md patterns that survive production
- Claude Code workflows for real teams
- Refactoring at scale (the parts AI helps and the parts it doesn't)
- TypeScript strict, Clean Architecture, DDD

No tool roundups. No AI hype. No listicles. If a tip can't survive a year of refactors, you won't read it.

**Subscribe (free): alexrogov.substack.com**

## About the author

**Alex Rogov** is a Full-Stack JS/TS Architect based in Amsterdam.

10+ years on TypeScript, Node.js, NestJS, React, React Native, Next.js. Now building production systems with Claude Code, CLAUDE.md, and AI agents in the loop.

- Newsletter: alexrogov.substack.com
- X: @Alex_Rogov_js
- LinkedIn: rogov-alex
- Hashnode: alexrogov.hashnode.dev
- GitHub: autosergach

© 2026 Alex Rogov. Forward to one teammate. That's how this content grows.